

Agentic AI: Foundations and Open-Source Practice

Day 1 Lab Manual

Python in VS Code: Chatbot vs Rule-Based Workflow vs AI Agent

1. Lab Details

Lab Title	Set up a Python environment in VS Code; compare a simple chatbot, a rule-based workflow, and an AI agent on the same task
Unit	Unit 1: Foundations of AI Agents (sub-topics 1.1 and 1.2)
Editor	Visual Studio Code with the Python extension
LLM options	A. Ollama with a very small local model (no key), or B. a free Groq API key, or C. a free Hugging Face token
Duration	About 1.5 to 2 hours
Output	A working project folder (day1_lab) with five Python programs and a completed observation table

2. Aim

To set up a Python development environment in VS Code, connect it to an open large language model, and implement and compare three systems on the same task: a plain LLM chatbot, a rule-based workflow, and a tool-using AI agent.

3. Objectives

After completing this lab, you will be able to:

1. Install VS Code with the Python extension and create a virtual environment inside it.
2. Store secrets safely in a .env file and load them in Python.
3. Connect Python to an LLM through an OpenAI-compatible API, using either a local model or a free cloud key.
4. Implement a plain LLM chatbot and observe hallucination when private data is missing.
5. Implement a rule-based workflow and observe its reliability and its rigidity.
6. Implement a tool-using AI agent and trace its reason, act, and observe steps.
7. Compare the three systems and justify which one suits a given real-world task.

4. Prerequisites

4.1 Knowledge

- Basic Python: variables, functions, dictionaries, loops, if/else, and importing modules.
- Running commands in a terminal.
- Day 1 theory: what an LLM is, and Agent = LLM + Tools + Loop.

4.2 Hardware

Item	If you use Ollama (Option A)	If you use Groq or Hugging Face (Options B and C)
RAM	8 GB or more	4 GB is enough
Free disk space	About 5 GB for the model	Under 1 GB
GPU	Not required	Not required
Internet	Once, to download the model	Needed every time you run the code

4.3 Software

Software	Purpose	License
Python 3.11 or newer	Programming language	PSF License
Visual Studio Code	Code editor and terminal	MIT (the Microsoft build adds a proprietary licence; VSCodium is the fully open build)
Python extension for VS Code	Runs and debugs Python inside VS Code	MIT
openai (Python package)	Client library for any OpenAI-compatible API	Apache 2.0
python-dotenv (Python package)	Loads settings and keys from the .env file	BSD
Ollama (only for Option A)	Runs open LLMs locally	MIT

5. Choose Your LLM Option

All three options use exactly the same Python code. Only the .env file changes.

	A. Ollama (local)	B. Groq (free key)	C. Hugging Face (free token)
Best for	Laptops with 8 GB RAM or more; works offline after download	Low-end laptops; fastest responses	Low-end laptops; many open models
Cost	Free	Free tier with rate limits	Free monthly credits
Sign-up	None	console.groq.com	huggingface.co
Model used here	qwen2.5:1.5b (about 1 GB, Apache 2.0)	openai/gpt-oss-20b (Apache 2.0 model)	openai/gpt-oss-20b (Apache 2.0 model)
Tool calling	Works, but a small model sometimes ignores tools	Reliable	Reliable
Data privacy	Everything stays on your laptop	Questions are sent to Groq	Questions are sent to the provider

Model names on cloud services change over time. If a model ID is rejected, open the provider's model list (console.groq.com/docs/models or huggingface.co/models) and use a current one that supports tool calling.

Do not use the qwen2.5:3b model in this course: it has a research licence rather than Apache 2.0. The 1.5B and 7B sizes are Apache 2.0.

6. Video Tutorials

Watch Videos 1 and 3 before the lab. The others are for revision and deeper learning.

No.	Title and Link	Use in this lab
1	Create Python Virtual Environment in VS Code (Step-by-Step) https://www.youtube.com/watch?v=Ocs7PBwcyml	Steps 1 to 4: VS Code, Python extension, and the .venv virtual environment.
2	Create Virtual Environment for Python in Visual Studio Code (VSCode Venv) https://www.youtube.com/watch?v=sPxpoxxGf3E	Alternative walkthrough of the same steps if the first video is unclear.
3	How to Install Ollama and Run Models Locally (2026) https://www.youtube.com/watch?v=tVTwZRhxw9w	Option A only: installing Ollama and pulling a small model.
4	Ollama Course – Build AI Apps Locally (freeCodeCamp.org) https://www.youtube.com/watch?v=GWb9ApTPTv4	Optional. Watch the setup and basic command sections; the full course is about 3 hours.
5	AI Agents, Clearly Explained (Jeff Su) https://www.youtube.com/watch?v=FwOTs4UxQS4	Concept revision: the difference between an LLM, a workflow, and an agent.

Installer screens change over time, so a video may not match today's version exactly. If they differ, follow the commands in this manual. If a link does not open, search the exact title on YouTube.

7. Problem Statement

A college has three courses with private fee data that no public LLM has seen: **CS101 = Rs. 12,000, AI202 = Rs. 18,000, DS303 = Rs. 15,000**. Three systems will answer the same four questions:

System	How it works
1. Chatbot	Sends the question straight to the LLM. No data, no tools.
2. Rule-based workflow	Fixed Python if/else rules written by the programmer. No LLM.
3. AI agent	The LLM decides which tools to call (fee lookup, calculator) in a loop until it can answer.

Test questions:

1. What is the fee for AI202? (Correct: Rs. 18,000)
2. What is the total fee for CS101 and AI202 after a 10% scholarship? (Correct: Rs. 27,000)

3. Is DS303 more expensive than CS101, and by how much? (Correct: Yes, by Rs. 3,000)
4. Write a two-line welcome message for new AI students. (No data needed)

8. Procedure: Part A – Set Up the Environment

Step 1: Install Python and VS Code

Download Python 3.11 or newer from <https://www.python.org/downloads/> and install it. On Windows, tick **"Add python.exe to PATH"**. Then download and install VS Code from <https://code.visualstudio.com/download>.

Step 2: Install the Python extension

Open VS Code, click the **Extensions** icon in the left bar (or press Ctrl+Shift+X), search for **Python** by Microsoft, and click Install.

Step 3: Create the project folder

Create a folder named **day1_lab** on your computer. In VS Code choose **File > Open Folder** and select it. Open the built-in terminal with **Ctrl+`** (View > Terminal) and check Python:

```
python --version      # Windows
python3 --version     # Linux / macOS
```

Step 4: Create and select the virtual environment

Press **Ctrl+Shift+P**, type **Python: Create Environment**, choose **Venv**, then choose your Python 3.11+ interpreter. VS Code creates a `.venv` folder and selects it automatically.

If you prefer the terminal, run these commands instead, then press Ctrl+Shift+P, choose **Python: Select Interpreter**, and pick the interpreter inside `.venv`:

```
python -m venv .venv

# Activate: Windows PowerShell
.venv\Scripts\Activate.ps1
# Activate: Windows Command Prompt
.venv\Scripts\activate
# Activate: Linux / macOS
source .venv/bin/activate
```

When the environment is active, the terminal prompt starts with **(.venv)**.

If PowerShell blocks activation with a script execution error, run this once in the same terminal:
`Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`

Step 5: Install the Python packages

Create a file named **requirements.txt** in the folder (File > New File) with this content:

```
openai>=1.40.0
python-dotenv>=1.0.0
```

Then install it in the terminal:

```
pip install -r requirements.txt
```

Step 6: Set up your LLM (choose ONE option)

Option A: Ollama with a small local model

Install Ollama from <https://ollama.com/download> (on Linux run the script below), then download a small model and test it:

```
# Linux only
curl -fsSL https://ollama.com/install.sh | sh

ollama --version
ollama pull qwen2.5:1.5b      # about 1 GB
ollama list                  # confirm the model is there
ollama run qwen2.5:1.5b      # type a question, then /bye to exit
```

Ollama keeps a local server running on port 11434. Confirm it answers:

```
curl http://localhost:11434/v1/models
```

Option B: Groq free API key

Go to <https://console.groq.com>, sign up (no credit card), open **API Keys**, click **Create API Key**, and copy the key immediately. It is shown only once.

Option C: Hugging Face free token

Go to <https://huggingface.co>, sign up, open **Settings > Access Tokens**, create a token with **Read** access (Inference Providers permission), and copy it.

Step 7: Create the .env file

Create a file named **.env** in the day1_lab folder. Keep only the lines for the option you chose:

```
# Choose ONE provider: ollama (local), groq (free key) or huggingface (free key)
PROVIDER=ollama

# Option A - Ollama (local, no key needed)
MODEL=qwen2.5:1.5b

# Option B - Groq: get a free key at https://console.groq.com/keys
# PROVIDER=groq
# GROQ_API_KEY=gsk_XXXXXXXXXXXXXXXXXXXXX
# MODEL=openai/gpt-oss-20b

# Option C - Hugging Face: get a free token at https://huggingface.co/settings/tokens
# PROVIDER=huggingface
# HF_TOKEN=hf_XXXXXXXXXXXXXXXXXXXXX
# MODEL=openai/gpt-oss-20b
```

Never upload the .env file to GitHub or share your key in WhatsApp or email. If a key leaks, delete it in the provider's console and create a new one. Create a file named .gitignore containing the single line .env before you push anything.

9. Procedure: Part B – Write and Run the Programs

Create each file below in the day1_lab folder. To run a file, open it and press the **Run** (triangle) button at the top right, or run it in the terminal with: `python <filename>`

Your folder should look like this:

```

day1_lab/
|- .env
|- .gitignore
|- requirements.txt
|- config.py      <- settings, provider and data
|- check_setup.py <- step 1: test the connection
|- chatbot.py     <- system 1
|- workflow.py    <- system 2
|- tools.py       <- tools used by the agent
|- agent.py       <- system 3
|- challenge.py   <- extra question
|- .venv/         <- created by VS Code

```

9.1 File: config.py

This file chooses the provider, creates the client, and holds the course data. Every other file imports from it.

```

"""Shared configuration: chooses the LLM provider and holds the lab data."""
import os
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv() # reads the .env file in this folder

PROVIDER = os.getenv("PROVIDER", "ollama").strip().lower()

if PROVIDER == "ollama":
    # Option A: local model, no key
    BASE_URL = "http://localhost:11434/v1"
    API_KEY = "ollama" # any text works for Ollama
    MODEL = os.getenv("MODEL", "qwen2.5:1.5b")
elif PROVIDER == "groq":
    # Option B: free cloud key
    BASE_URL = "https://api.groq.com/openai/v1"
    API_KEY = os.getenv("GROQ_API_KEY")
    MODEL = os.getenv("MODEL", "openai/gpt-oss-20b")
elif PROVIDER == "huggingface":
    # Option C: free cloud key
    BASE_URL = "https://router.huggingface.co/v1"
    API_KEY = os.getenv("HF_TOKEN")
    MODEL = os.getenv("MODEL", "openai/gpt-oss-20b")
else:
    raise SystemExit(f"Unknown PROVIDER '{PROVIDER}'. Use ollama, groq or huggingface.")

if not API_KEY:
    raise SystemExit(f"No API key found for PROVIDER={PROVIDER}. Check your .env file.")

client = OpenAI(base_url=BASE_URL, api_key=API_KEY)

# Private college data that no public LLM has ever seen
COURSE_FEES = {"CS101": 12000, "AI202": 18000, "DS303": 15000}

QUESTIONS = [
    "What is the fee for AI202?",
    "What is the total fee for CS101 and AI202 after a 10% scholarship?",
    "Is DS303 more expensive than CS101, and by how much?",
    "Write a two-line welcome message for new AI students.",
]

def banner(system_name):
    print(f"\n=== {system_name} | provider: {PROVIDER} | model: {MODEL} ===\n")

```

9.2 File: check_setup.py (run this first)

```

"""Step 1: run this first. It checks the environment and the model connection."""
import sys
from config import client, MODEL, PROVIDER

print("Python version :", sys.version.split()[0])
print("Provider       :", PROVIDER)

```

```

print("Model      :", MODEL)
print("Calling the model ...")

response = client.chat.completions.create(
    model=MODEL,
    messages=[{"role": "user", "content": "Reply with exactly these two words: SETUP
OK"}],
    temperature=0,
)
print("Model replied :", response.choices[0].message.content.strip())
print("\nSetup check finished.")

```

Run it: python check_setup.py

Expected output (version numbers and the model name will differ):

```

Python version : 3.12.3
Provider       : ollama
Model          : qwen2.5:1.5b
Calling the model ...
Model replied  : SETUP OK

Setup check finished.

```

If you see this, your environment, packages, key, and model all work. Do not continue until this passes; see the troubleshooting table in Section 12.

9.3 File: chatbot.py (System 1)

```

"""System 1: a plain LLM chatbot. No tools, no access to the college data."""
from config import client, MODEL, QUESTIONS, banner

def chatbot(question):
    response = client.chat.completions.create(
        model=MODEL,
        messages=[
            {"role": "system", "content": "You are a helpful college assistant."},
            {"role": "user", "content": question},
        ],
        temperature=0,
    )
    return response.choices[0].message.content.strip()

if __name__ == "__main__":
    banner("SYSTEM 1: CHATBOT")
    for question in QUESTIONS:
        print("Q:", question)
        print("A:", chatbot(question))
        print("-" * 70)

```

Run it: python chatbot.py

Sample output (wording will differ on every run and every model):

```

=== SYSTEM 1: CHATBOT | provider: ollama | model: qwen2.5:1.5b ===

Q: What is the fee for AI202?
A: The fee for AI202 is approximately Rs. 25,000 per semester.
-----
Q: What is the total fee for CS101 and AI202 after a 10% scholarship?
A: Assuming each course costs Rs. 20,000, the total after a 10% scholarship
   would be Rs. 36,000.
-----
Q: Is DS303 more expensive than CS101, and by how much?
A: DS303 is generally more expensive than CS101 by around Rs. 5,000.
-----
Q: Write a two-line welcome message for new AI students.

```

```
A: Welcome to the world of Artificial Intelligence!
    May your curiosity lead you to build something remarkable.
-----
```

Correctness check: questions 1 to 3 are **wrong**, because the model has never seen your fee data. Question 4 is answered well, because it needs no data. Some models instead reply that they do not have the fee list, which is also acceptable behaviour. Note down which one your model did.

Observe: the wrong answers are written confidently, with no warning. This is hallucination.

9.4 File: workflow.py (System 2)

```
"""System 2: a rule-based workflow. Fixed if/else rules, no LLM at all."""
import re
from config import COURSE_FEES, QUESTIONS

def workflow(question):
    codes = re.findall(r"[A-Z]{2}\d{3}", question.upper())
    fees = [COURSE_FEES[code] for code in codes if code in COURSE_FEES]

    if not fees:
        return "Sorry, I can only answer questions about course fees."

    text = question.lower()
    if "total" in text:
        total = sum(fees)
        percent = re.search(r"(\d+)\s*", text)
        if "scholarship" in text and percent:
            total = total * (1 - int(percent.group(1)) / 100)
        return f"Total fee: Rs. {total:,.0f}"

    if len(fees) == 1:
        return f"Fee for {codes[0]}: Rs. {fees[0]:,}"

    return "Sorry, I do not have a rule for this type of question."

if __name__ == "__main__":
    print("\n=== SYSTEM 2: RULE-BASED WORKFLOW (no LLM) ===\n")
    for question in QUESTIONS:
        print("Q:", question)
        print("A:", workflow(question))
        print("-" * 70)
```

Run it: python workflow.py

Expected output (exactly this, every time, on every machine):

```
=== SYSTEM 2: RULE-BASED WORKFLOW (no LLM) ===

Q: What is the fee for AI202?
A: Fee for AI202: Rs. 18,000
-----
Q: What is the total fee for CS101 and AI202 after a 10% scholarship?
A: Total fee: Rs. 27,000
-----
Q: Is DS303 more expensive than CS101, and by how much?
A: Sorry, I do not have a rule for this type of question.
-----
Q: Write a two-line welcome message for new AI students.
A: Sorry, I can only answer questions about course fees.
-----
```


Correctness check: questions 1 and 2 are exactly right and instant. Questions 3 and 4 fail, because no rule was written for them. This output was verified on a test machine, so if yours differs, compare your code line by line.

Try this: rephrase question 2 as "How much will I pay for CS101 plus AI202 with a 10 percent scholarship?" and run again. The workflow now misses the word "total", so the rule does not fire. Rigid rules break on new wording.

9.5 File: tools.py (tools for the agent)

These are the functions the agent is allowed to run, plus the JSON Schema descriptions the LLM reads when choosing a tool.

```
"""Tools the agent is allowed to use, plus their JSON Schema descriptions."""
import ast
import operator
from config import COURSE_FEES

def get_course_fee(course_code: str) -> str:
    """Look up the fee for one course code."""
    fee = COURSE_FEES.get(course_code.strip().upper())
    return str(fee) if fee is not None else f"Unknown course code: {course_code}"

# A safe calculator: only numbers and + - * / ( ) are allowed. Never use eval().
_OPS = {ast.Add: operator.add, ast.Sub: operator.sub, ast.Mult: operator.mul,
        ast.Div: operator.truediv, ast.USub: operator.neg}

def _evaluate(node):
    if isinstance(node, ast.Constant) and isinstance(node.value, (int, float)):
        return node.value
    if isinstance(node, ast.BinOp) and type(node.op) in _OPS:
        return _OPS[type(node.op)](_evaluate(node.left), _evaluate(node.right))
    if isinstance(node, ast.UnaryOp) and type(node.op) in _OPS:
        return _OPS[type(node.op)](_evaluate(node.operand))
    raise ValueError("Unsupported expression")

def calculator(expression: str) -> str:
    """Evaluate a basic arithmetic expression such as (12000 + 18000) * 0.9."""
    try:
        return str(_evaluate(ast.parse(expression, mode="eval").body))
    except Exception as error:
        return f"Calculator error: {error}"

TOOL_FUNCTIONS = {"get_course_fee": get_course_fee, "calculator": calculator}

# These descriptions are what the LLM reads when deciding which tool to call
TOOLS = [
    {"type": "function", "function": {
        "name": "get_course_fee",
        "description": "Get the fee in rupees for a single course code, for example CS101.",
        "parameters": {"type": "object",
            "properties": {"course_code": {"type": "string"}},
            "required": ["course_code"]}},
    {"type": "function", "function": {
        "name": "calculator",
        "description": "Evaluate an arithmetic expression using + - * / and brackets.",
        "parameters": {"type": "object",
            "properties": {"expression": {"type": "string"}},
            "required": ["expression"]}}}
]

if __name__ == "__main__":
    print("get_course_fee('ai202') ->", get_course_fee("ai202"))
    print("calculator('(12000 + 18000) * 0.9') ->", calculator("(12000 + 18000) * 0.9"))
    print("calculator('15000 - 12000') ->", calculator("15000 - 12000"))
```

Run it: python tools.py

Expected output (exactly this):

```
get_course_fee('ai202') -> 18000
calculator('(12000 + 18000) * 0.9') -> 27000.0
calculator('15000 - 12000') -> 3000
```

9.6 File: agent.py (System 3)

```
"""System 3: an AI agent. LLM + tools + loop."""
import json
from config import client, MODEL, QUESTIONS, banner
from tools import TOOLS, TOOL_FUNCTIONS

SYSTEM_PROMPT = (
    "You are a college fee assistant. Never guess a fee: always use get_course_fee. "
    "Use calculator for any arithmetic. Available course codes: CS101, AI202, DS303. "
    "If no tool is needed, answer directly."
)

def agent(question, max_steps=6, verbose=True):
    messages = [{"role": "system", "content": SYSTEM_PROMPT},
                 {"role": "user", "content": question}]

    for step in range(1, max_steps + 1):
        # 1. REASON: ask the LLM what to do next
        response = client.chat.completions.create(
            model=MODEL, messages=messages, tools=TOOLS, temperature=0)
        message = response.choices[0].message

        # 2. If no tool is requested, the LLM has finished
        if not message.tool_calls:
            return message.content.strip()

        messages.append({
            "role": "assistant", "content": message.content or "",
            "tool_calls": [{"id": call.id, "type": "function",
                           "function": {"name": call.function.name,
                                         "arguments": call.function.arguments}}
                           for call in message.tool_calls]})

        # 3. ACT and OBSERVE: run each tool, send the result back
        for call in message.tool_calls:
            name = call.function.name
            arguments = json.loads(call.function.arguments or "{}")
            function = TOOL_FUNCTIONS.get(name)
            result = function(**arguments) if function else f"Unknown tool: {name}"
            if verbose:
                print(f"    step {step}: {name}({arguments}) -> {result}")
            messages.append({"role": "tool", "tool_call_id": call.id, "content": result})

    return "Stopped: maximum steps reached without a final answer."

if __name__ == "__main__":
    banner("SYSTEM 3: AI AGENT")
    for question in QUESTIONS:
        print("Q:", question)
        print("A:", agent(question))
        print("-" * 70)
```

Run it: python agent.py

Sample output (the tool order and the wording vary between runs):

```
=== SYSTEM 3: AI AGENT | provider: ollama | model: qwen2.5:1.5b ===

Q: What is the fee for AI202?
```

```

    step 1: get_course_fee({'course_code': 'AI202'}) -> 18000
A: The fee for AI202 is Rs. 18,000.
-----
Q: What is the total fee for CS101 and AI202 after a 10% scholarship?
    step 1: get_course_fee({'course_code': 'CS101'}) -> 12000
    step 1: get_course_fee({'course_code': 'AI202'}) -> 18000
    step 2: calculator({'expression': '(12000 + 18000) * 0.9'}) -> 27000.0
A: The total fee after a 10% scholarship is Rs. 27,000.
-----
Q: Is DS303 more expensive than CS101, and by how much?
    step 1: get_course_fee({'course_code': 'DS303'}) -> 15000
    step 1: get_course_fee({'course_code': 'CS101'}) -> 12000
    step 2: calculator({'expression': '15000 - 12000'}) -> 3000
A: Yes. DS303 costs Rs. 3,000 more than CS101.
-----
Q: Write a two-line welcome message for new AI students.
A: Welcome to the AI programme! Ask questions, build often,
    and enjoy every experiment.
-----

```

Correctness check: the agent should reach Rs. 27,000 for question 2 and Rs. 3,000 for question 3, and it should call **no tool at all** for question 4. The printed step lines prove the reasoning path: this is the agent loop working.

Small local models sometimes skip a tool and guess, or repeat the same call. If that happens, run it again, or switch to Groq or Hugging Face in .env (Option B or C), or use ollama pull qwen2.5:7b if your laptop has 16 GB RAM. Note the failure in your observations: unreliable tool use is a real agent problem, studied further on Day 3 and Day 6.

9.7 File: challenge.py (a question nobody designed for)

```

"""A question none of the three systems was designed for."""
from workflow import workflow
from agent import agent

QUESTION = "I can pay Rs. 30,000. Which two courses can I take together within this budget?"

print("Q:", QUESTION)
print("\nWorkflow :", workflow(QUESTION))
print("\nAgent      :", agent(QUESTION))

```

Run it: python challenge.py

What to expect: the workflow replies that it has no rule for this. The agent looks up the fees and uses the calculator to test pairs. The only pair within Rs. 30,000 is **CS101 + DS303 = Rs. 27,000**. A small model may get this wrong; a larger one usually gets it right.

This is the key lesson of the lab: the workflow is reliable but rigid, and the agent is flexible but less predictable.

10. Observations

Fill in this table from your own runs.

Criterion	Chatbot	Workflow	Agent
Q1 correct? (Y/N)			
Q2 correct? (Y/N)			

Criterion	Chatbot	Workflow	Agent
Q3 correct? (Y/N)			
Q4 handled well? (Y/N)			
Challenge question handled? (Y/N)			
Same output on a repeat run? (Y/N)			
Approximate response time			
Number of LLM calls per question			
One strength			
One weakness			
Best suited for (one real use case)			

Agent trace: copy the printed steps for Question 2.

Step	Tool called and arguments	Result (observation)
1		
2		
3		
4		

11. Discussion Questions

1. The chatbot gave a confident but wrong fee. Why is that more dangerous than replying "I don't know"?
2. The workflow was always correct for questions 1 and 2. Why might a finance office still prefer it over the agent?
3. The agent's steps can change between runs. What problems would that cause in a real product?
4. Design a system that uses a workflow for common questions and an agent for the rest. Where would you draw the line?
5. Which parts of agent.py are the LLM, the tools, and the loop?

12. Troubleshooting

Problem	Solution
ModuleNotFoundError: No module named 'openai' or 'dotenv'	The virtual environment is not selected. Press Ctrl+Shift+P > Python: Select Interpreter > choose the one inside .venv, open a new terminal, and run pip install -r requirements.txt again.
'python' is not recognized (Windows)	Python was not added to PATH. Reinstall Python with the PATH box ticked, or use the py command instead.

Problem	Solution
PowerShell: running scripts is disabled	Run once in that terminal: Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
APIConnectionError / Connection refused on port 11434	Ollama is not running. Start the Ollama app, or run ollama serve in a separate terminal.
model 'qwen2.5:1.5b' not found	Run ollama pull qwen2.5:1.5b, then ollama list to confirm. Or change MODEL in .env to a model you already have.
401 Unauthorized / invalid api key	The key in .env is wrong, has extra spaces or quotes, or was deleted. Create a new key and paste it again.
404 model_not_found or model_decommissioned (Groq / Hugging Face)	That model ID no longer exists. Pick a current tool-calling model from console.groq.com/docs/models or huggingface.co/models and update MODEL in .env.
429 rate limit exceeded	Free tiers limit requests per minute. Wait a minute and run again, or switch to Option A (Ollama).
No API key found for PROVIDER=groq	The .env file is missing, is in the wrong folder, or the variable name is misspelled. It must sit inside day1_lab next to config.py.
The agent answers with a guessed fee instead of calling a tool	The small model ignored the tools. Re-run, or switch to Option B or C, or use a bigger local model. Record it as an agent failure mode.
json.JSONDecodeError inside agent.py	The model produced malformed tool arguments. Re-run; robust handling of this case is covered on Day 6.
Stopped: maximum steps reached	The agent looped without finishing. Note the printed steps; this is the max_steps safety limit doing its job.
Very slow answers with Ollama	Use Option B or C, close other applications, or keep to the 1.5B model.

13. Viva Questions

1. What is the difference between a chatbot, a rule-based workflow, and an AI agent?
2. In agent.py, which lines are the LLM, which are the tools, and which is the loop?
3. Who actually executes a tool: the LLM or your Python program?
4. Why does the agent need a max_steps limit?
5. Why does the calculator tool avoid Python's eval() function?
6. What is the purpose of the JSON Schema tool descriptions in tools.py?
7. Why do we use a virtual environment for each project?
8. Why is the API key kept in a .env file instead of inside config.py?
9. What does it mean that Ollama, Groq, and Hugging Face all provide an OpenAI-compatible API?

14. Result

Thus, a Python environment was set up in VS Code and connected to an open large language model, and a chatbot, a rule-based workflow, and an AI agent were implemented and compared on the same task. The observations show that
